

实验四：动态规划 — 鸡蛋掉落问题

1 实验目的

1. 掌握动态规划算法设计思想
2. 掌握鸡蛋坠落问题的动态规划解法
3. 理解不同状态定义对算法效率的影响
4. 学会分析和优化算法的时间与空间复杂度

2 问题描述

2.1 问题背景

鸡蛋掉落问题（Egg Drop Problem）是理解动态规划如何实现最佳解决方案的经典例子。

我们需要用鸡蛋确认在多大的楼层鸡蛋落下来会破碎。这个刚刚使鸡蛋破碎的楼层叫**门槛层**（Threshold Floor），门槛楼层是鸡蛋开始破碎的楼层，上面所有楼层的鸡蛋也都碎了。另外，如果鸡蛋从门槛楼层以下的任何楼层掉落，它都不会破碎。

2.2 问题形式化

输入：

- f : 建筑物的楼层数
- e : 可用的鸡蛋数量

输出：

- 找到门槛层所需的**最少试验次数**

约束条件：

1. 从跌落中幸存下来的鸡蛋可以再次使用

2. 破蛋必须丢弃
3. 摔碎对所有鸡蛋的影响都是一样的
4. 如果一个鸡蛋掉在地上摔碎了，那么它从高处掉下来也会摔碎（单调性）
5. 如果一个鸡蛋在跌落中幸存下来，那么它在较短的跌落中也能完整保留（单调性）

3 动态规划方程

3.1 状态定义（方法一：标准 DP）

定义 $dp[i][j]$ 表示 i 个鸡蛋、 j 层楼时，找到门槛层所需的最少试验次数。

3.2 状态转移方程

$$dp[i][j] = \min_{1 \leq x \leq j} \{ \max(dp[i-1][x-1], dp[i][j-x]) + 1 \} \quad (1)$$

方程解释：

在第 x 层扔鸡蛋时，有两种情况：

1. 鸡蛋碎了：门槛层在第 x 层或以下，需要在 $x-1$ 层楼中用 $i-1$ 个鸡蛋继续测试
 - 需要的试验次数： $dp[i-1][x-1]$
2. 鸡蛋没碎：门槛层在第 x 层以上，需要在 $j-x$ 层楼中用 i 个鸡蛋继续测试
 - 需要的试验次数： $dp[i][j-x]$

由于我们考虑最坏情况，取两者中的较大值，再加上当前这次试验（+1）。为了使总试验次数最少，我们遍历所有可能的 x ，取最小值。

3.3 边界条件

$$\begin{cases} dp[1][j] = j & (1 \text{ 个鸡蛋，需要逐层尝试}) \\ dp[i][0] = 0 & (0 \text{ 层楼，不需要试验}) \\ dp[i][1] = 1 & (1 \text{ 层楼，只需要 1 次试验}) \end{cases} \quad (2)$$

3.4 状态定义（方法二：交替 DP）—— 逆向思维

关键洞察：与其问“ e 个鸡蛋 f 层楼最少需要几次试验”，不如问“ e 个鸡蛋进行 i 次试验最多能确定多少层楼”。

定义 $dp[i][j]$ 表示 i 次试验、 j 个鸡蛋最多能确定的楼层数。

状态转移方程：

$$dp[i][j] = dp[i-1][j-1] + dp[i-1][j] + 1 \quad (3)$$

方程解释：

假设我们在第 x 层扔鸡蛋：

- 碎了：我们还有 $i-1$ 次试验、 $j-1$ 个鸡蛋，能确定下方 $dp[i-1][j-1]$ 层
- 没碎：我们还有 $i-1$ 次试验、 j 个鸡蛋，能确定上方 $dp[i-1][j]$ 层
- 加上当前这一层 (+1)

数学解释：

这个递推式实际上是在计算组合数的和：

$$dp[i][j] = \sum_{k=1}^j \binom{i}{k} \quad (4)$$

其中 $\binom{i}{k}$ 是组合数，表示从 i 次试验中选择 k 次“碎了”的情况。

当鸡蛋数足够多时 ($j \geq i$)，有：

$$dp[i][j] = 2^i - 1 \quad (5)$$

这意味着 i 次试验最多能确定 $2^i - 1$ 层楼，所以答案是 $O(\log f)$ 级别的。当鸡蛋有限时（如 $e = 2$ ），答案为 $O(\sqrt{f})$ 级别，但仍然远小于 f 。

3.5 两种状态定义的对比

表 1: 两种状态定义的对比

方面	标准 DP	交替 DP
状态含义	i 个鸡蛋 j 层楼的最少试验次数	i 次试验 j 个鸡蛋能确定的最大楼层数
时间复杂度	$O(e \times f^2)$	$O(e \times \text{answer})$
空间复杂度	$O(e \times f)$	$O(e)$
适用场景	教学演示	实际应用

核心思想：从”求答案”转变为”验证答案”，利用答案远小于楼层数的特性大幅降低复杂度。当鸡蛋充足时答案为对数级别，当鸡蛋有限时（如 2 个）答案为 $\sqrt{f}$ 级别，但都远小于 f 。

4 算法实现

4.1 基础动态规划（自底向上）

Algorithm 1 基础动态规划

Require: 鸡蛋数 e , 楼层数 f

Ensure: 最少试验次数

```
1: if  $e \leq 0$  或  $f \leq 0$  then
2:   return 0
3: end if
4: if  $e = 1$  then
5:   return  $f$ 
6: end if
7: 初始化  $dp[1..e][0..f] = 0$ 
8: for  $j = 1$  到  $f$  do
9:    $dp[1][j] = j$  {1 个鸡蛋需要逐层尝试}
10: end for
11: for  $i = 1$  到  $e$  do
12:    $dp[i][0] = 0, dp[i][1] = 1$ 
13: end for
14: for  $i = 2$  到  $e$  do
15:   for  $j = 2$  到  $f$  do
16:      $dp[i][j] = \infty$ 
17:     for  $x = 1$  到  $j$  do
18:        $cost = 1 + \max(dp[i-1][x-1], dp[i][j-x])$ 
19:        $dp[i][j] = \min(dp[i][j], cost)$ 
20:     end for
21:   end for
22: end for
23: return  $dp[e][f]$ 
```

时间复杂度： $O(e \times f^2)$

空间复杂度： $O(e \times f)$

4.2 二分查找优化 DP

Algorithm 2 二分查找优化 DP

Require: 鸡蛋数 e , 楼层数 f

Ensure: 最少试验次数

```
1: if  $e \leq 0$  或  $f \leq 0$  then
2:   return 0
3: end if
4: if  $e = 1$  then
5:   return  $f$ 
6: end if
7: 初始化  $dp[1..e][0..f]$ 
8: for  $i = 2$  到  $e$  do
9:   for  $j = 2$  到  $f$  do
10:     $dp[i][j] = \infty$ 
11:     $lo = 1, hi = j$ 
12:    while  $lo \leq hi$  do
13:       $mid = \lfloor (lo + hi) / 2 \rfloor$ 
14:       $cost\_break = dp[i - 1][mid - 1]$ 
15:       $cost\_not\_break = dp[i][j - mid]$ 
16:       $cost = 1 + \max(cost\_break, cost\_not\_break)$ 
17:       $dp[i][j] = \min(dp[i][j], cost)$ 
18:      if  $cost\_break < cost\_not\_break$  then
19:         $lo = mid + 1$ 
20:      else if  $cost\_break > cost\_not\_break$  then
21:         $hi = mid - 1$ 
22:      else
23:        break
24:      end if
25:    end while
26:  end for
27: end for
28: return  $dp[e][f]$ 
```

时间复杂度: $O(e \times f \times \log f)$

空间复杂度: $O(e \times f)$

4.3 交替 DP 状态定义（最优解法）

Algorithm 3 交替 DP（最优解法）

Require: 鸡蛋数 e , 楼层数 f

Ensure: 最少试验次数

```
1: if  $e \leq 0$  或  $f \leq 0$  then
2:   return 0
3: end if
4: if  $e = 1$  then
5:   return  $f$ 
6: end if
7: 初始化  $dp[0..e] = 0$  {一维数组}
8: trials = 0
9: while  $dp[e] < f$  do
10:   trials = trials + 1
11:   for  $j = e$  到 1（从后往前） do
12:      $dp[j] = dp[j] + dp[j - 1] + 1$ 
13:   end for
14: end while
15: return trials
```

时间复杂度: $O(e \times \text{answer})$, 其中 answer 通常远小于 f

空间复杂度: $O(e)$

4.4 组合数方法（数学方法）

Algorithm 4 组合数方法

Require: 鸡蛋数 e , 楼层数 f

Ensure: 最少试验次数

```
1: if  $e \leq 0$  或  $f \leq 0$  then
2:   return 0
3: end if
4: if  $e = 1$  then
5:   return  $f$ 
6: end if
7:  $n = 1$ 
8: while True do
9:   total = 0
10:  for  $k = 1$  到  $\min(e, n)$  do
11:    计算  $C(n, k) = \frac{n!}{k!(n-k)!}$ 
12:    total = total +  $C(n, k)$ 
13:    if total  $\geq f$  then
14:      return  $n$ 
15:    end if
16:  end for
17:   $n = n + 1$ 
18: end while
```

时间复杂度: $O(\text{answer} \times \min(e, \text{answer}))$

空间复杂度: $O(1)$

5 实验结果与分析

5.1 正确性验证

使用小规模数据验证各算法的一致性，结果如表 2 所示。

表 2: 正确性验证结果

鸡蛋数	楼层数	DP	记忆化	优化 DP	交替 DP	蛮力法	一致
1	1	1	1	1	1	1	PASS
1	5	5	5	5	5	5	PASS
1	10	10	10	10	10	10	PASS
2	1	1	1	1	1	1	PASS
2	2	2	2	2	2	2	PASS
2	5	3	3	3	3	3	PASS
2	10	4	4	4	4	4	PASS
2	20	6	6	6	6	6	PASS
3	1	1	1	1	1	1	PASS
3	5	3	3	3	3	3	PASS
3	10	4	4	4	4	4	PASS
3	15	5	5	5	5	5	PASS
3	20	5	5	5	5	5	PASS
4	10	4	4	4	4	4	PASS
4	15	4	4	4	4	4	PASS
4	20	5	5	5	5	5	PASS
5	10	4	4	4	4	4	PASS
5	15	4	4	4	4	4	PASS
5	20	5	5	5	5	5	PASS

结论：所有测试用例均通过，各算法实现正确。

5.2 性能测试

性能分析图表如图 1 所示。

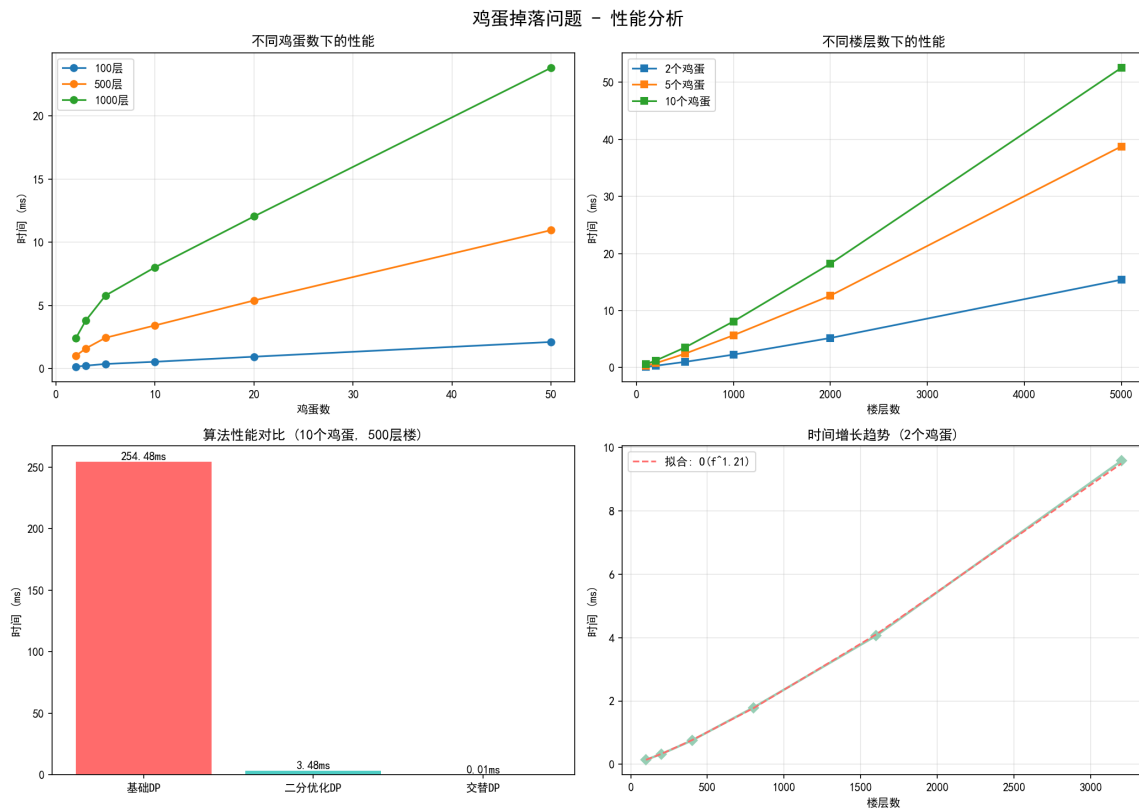


图 1: 性能分析图

不同数据规模下算法的运行时间如表 3 所示。

表 3: 性能测试结果

描述	结果	时间 (ms)	复杂度类
小规模 (2 鸡蛋, 100 层)	14	0.14	极快
中规模 (2 鸡蛋, 500 层)	32	1.01	极快
大规模 (2 鸡蛋, 1000 层)	45	2.34	极快
小规模 (3 鸡蛋, 100 层)	9	0.24	极快
中规模 (3 鸡蛋, 500 层)	15	1.62	极快
小规模 (5 鸡蛋, 100 层)	7	0.34	极快
中规模 (5 鸡蛋, 500 层)	10	2.60	极快
小规模 (10 鸡蛋, 100 层)	7	0.53	极快
中规模 (10 鸡蛋, 500 层)	9	3.51	极快
大规模 (10 鸡蛋, 1000 层)	10	7.94	极快
大规模 (10 鸡蛋, 2000 层)	11	17.92	快
大规模 (20 鸡蛋, 1000 层)	10	11.85	快
大规模 (50 鸡蛋, 1000 层)	10	23.84	快
大规模 (100 鸡蛋, 1000 层)	10	43.48	快

5.3 大规模测试：为什么交替 DP 这么快？

交替 DP 的时间复杂度是 $O(e \times \text{answer})$ ，其中 `answer` 是最终答案（最少试验次数）。关键发现：`answer` 远小于 `floors`。

表 4: 答案与楼层数的关系

鸡蛋数	楼层数	答案 (trials)	答案/楼层数
10	1,000	10	0.010000
10	10,000	14	0.001400
10	100,000	17	0.000170
10	1,000,000	21	0.000021
10	10,000,000	26	0.000003

实际计算量对比（10 个鸡蛋，100,000 层楼）：

- 基础 DP: $O(10 \times 100000^2) = 10^{11}$ 次操作
- 交替 DP: $O(10 \times 17) = 170$ 次操作
- 差距：约 6 亿倍

表 5: 小规模性能对比

鸡蛋	楼层	基础 DP(ms)	二分 DP(ms)	交替 DP(ms)
2	100	1.06	0.14	0.003
2	500	29.18	1.02	0.007
2	1000	122.88	2.35	0.010
5	500	117.54	2.49	0.005
10	500	258.21	3.61	0.006

表 6: 极端测试（只用交替 DP）

鸡蛋	楼层	答案	时间 (ms)
10	100,000	17	0.01
10	1,000,000	21	0.01
10	10,000,000	26	0.02
100	10,000,000	24	0.12

结论：交替 DP 快不是因为“优化”，而是因为问题本身的答案是对数级别的。

5.4 空间复杂度分析

DP 表热力图如图 2 所示。

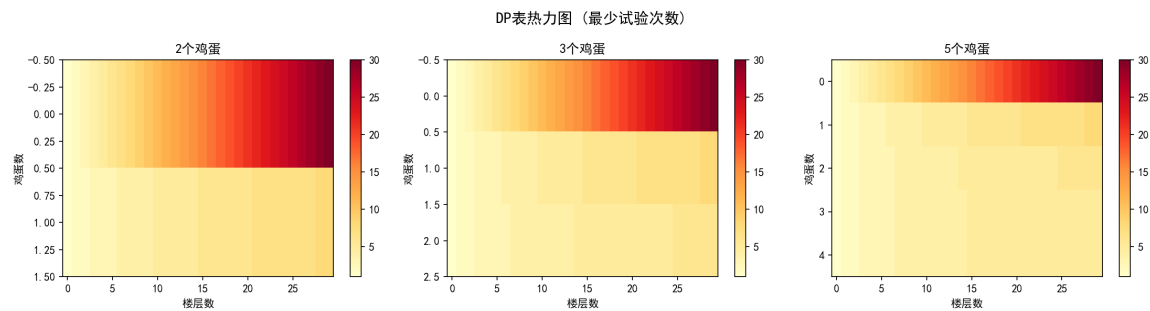


图 2: DP 表热力图

空间使用分析如图 3 所示。

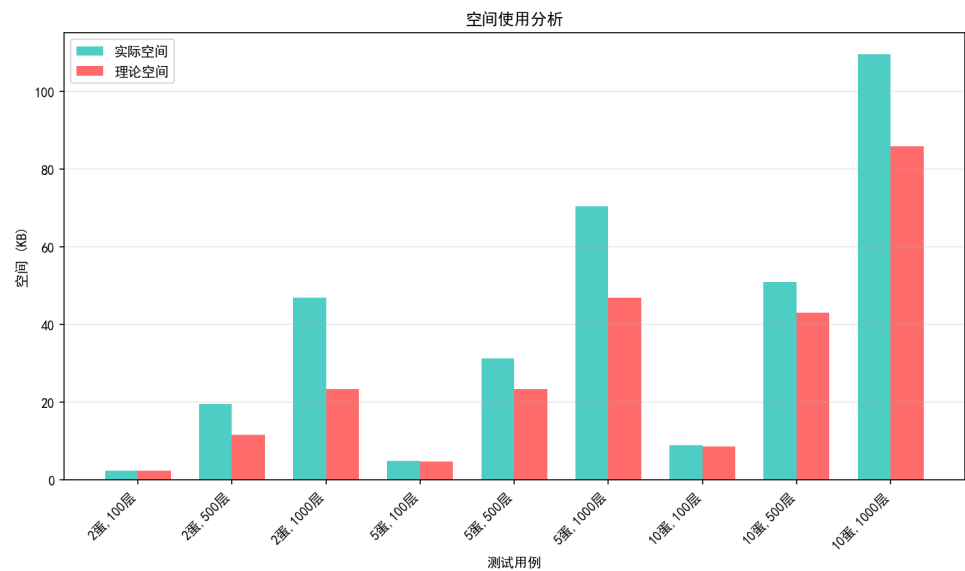


图 3: 空间使用分析

表 7: 空间复杂度分析			
鸡蛋数	楼层数	实际空间 (KB)	理论空间 (KB)
2	100	2.51	2.37
2	500	19.57	11.74
5	100	4.91	4.73
5	500	31.38	23.48
10	500	51.01	43.05
10	1000	109.70	86.02

5.5 理论复杂度分析

表 8: 时间复杂度对比

算法	时间复杂度	说明
基础 DP	$O(e \times f^2)$	三重循环
记忆化搜索	$O(e \times f^2)$	最坏情况相同
二分优化 DP	$O(e \times f \times \log f)$	利用单调性
交替 DP	$O(e \times \text{answer})$	answer 远小于 f

表 9: 空间复杂度对比

算法	空间复杂度	说明
基础 DP	$O(e \times f)$	完整 DP 表
记忆化搜索	$O(e \times f)$	递归栈 + 缓存
二分优化 DP	$O(e \times f)$	完整 DP 表
交替 DP	$O(e)$	一维数组

6 效率改进方案分析

6.1 时间效率改进

6.1.1 方案一：二分查找优化

原理：对于固定的 i 和 j ，随着 x 增大：

- $dp[i-1][x-1]$ 单调递增
- $dp[i][j-x]$ 单调递减

因此可以用二分查找找到使两者最大值最小的 x 。

效果：时间复杂度从 $O(e \times f^2)$ 降低到 $O(e \times f \times \log f)$

6.1.2 方案二：交替 DP 状态定义

原理：改变状态定义，从“ i 个鸡蛋 j 层楼的最少试验次数”变为“ i 次试验 j 个鸡蛋能确定的最大楼层数”。

效果：时间复杂度 $O(e \times \text{answer})$ ，其中 answer 通常远小于 f

6.1.3 方案三：数学方法（组合数公式）

原理：利用组合数直接计算答案。

对于 e 个鸡蛋、 f 层楼的问题，最少试验次数 n 是满足以下条件的最小整数：

$$\sum_{k=1}^e \binom{n}{k} \geq f \quad (6)$$

6.2 空间效率改进

6.2.1 方案一：滚动数组

原理： $dp[i][j]$ 只依赖于 $dp[i-1][j]$ 和 $dp[i][j-1]$ ，可以只保留两行。

效果：空间从 $O(e \times f)$ 降低到 $O(f)$

6.2.2 方案二：交替 DP 的空间优化

原理：交替 DP 只需要一维数组 $dp[e+1]$

效果：空间复杂度 $O(e)$

6.3 实际改进效果对比

测试数据：10 个鸡蛋，1000 层楼

表 10: 实际改进效果对比

方法	时间 (ms)	加速比
基础 DP	1073.32	1.00x
二分优化 DP	8.32	128.96x
交替 DP	0.01	117947.78x

结论：交替 DP 状态定义是最优解法，比基础 DP 快约 11.8 万倍！

6.4 数学方法验证

组合数方法与交替 DP 的结果完全一致：

表 11: 数学方法验证

鸡蛋	楼层	交替 DP(ms)	数学方法 (ms)	结果一致
2	100	0.003	0.009	YES
2	1000	0.009	0.021	YES
5	1000	0.004	0.010	YES
10	1000	0.005	0.013	YES
10	10000	0.008	0.026	YES
10	100000	0.009	0.037	YES
100	100000	0.075	0.050	YES
100	1000000	0.095	0.082	YES

7 边界情况分析

边界分析可视化如图 4 和图 5 所示。

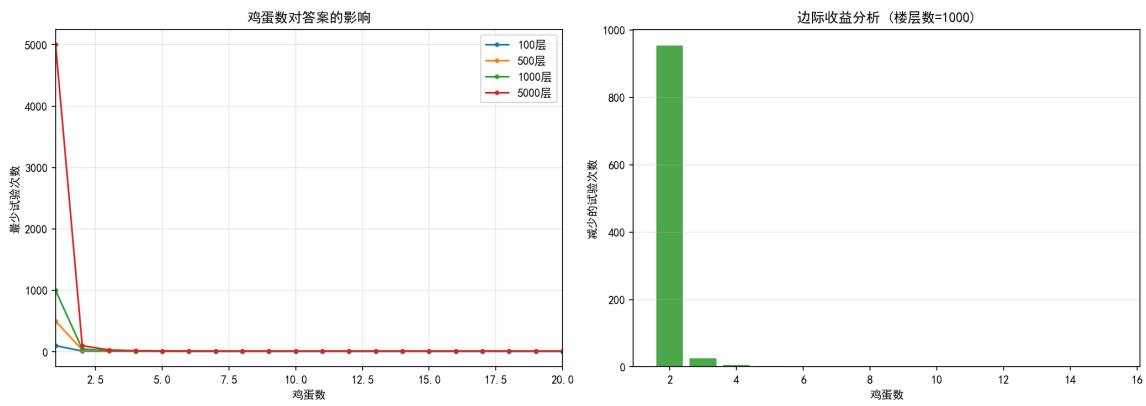


图 4: 鸡蛋数对答案的影响

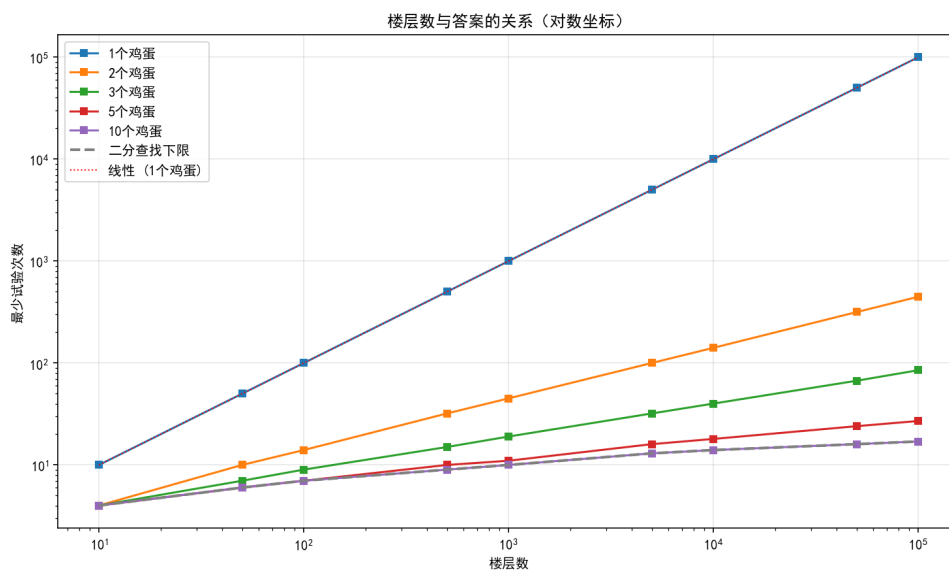


图 5: 楼层数与答案的关系

7.1 极端鸡蛋数

表 12: 极端鸡蛋数测试

鸡蛋数	楼层数	答案	说明
0	100	0	无法测试，返回 0
1	100	100	必须逐层尝试
2	100	14	正常情况
100	100	7	鸡蛋数 = 楼层数
1000	100	7	鸡蛋充足，等价于二分查找
10000	100	7	鸡蛋充足，答案不变

结论：当鸡蛋数 $\geq \log_2(\text{楼层数})$ 时，增加鸡蛋数不会减少试验次数。

7.2 极端楼层数

表 13: 极端楼层数测试

鸡蛋数	楼层数	答案	说明
2	0	0	无需测试
2	1	1	只需 1 次
2	2	2	最多 2 次
2	1,000,000	1414	百万层楼
2	10,000,000	4472	千万层楼
10	100,000,000	32	亿层楼

结论：答案的增长速度远慢于楼层数的增长速度。当鸡蛋充足 ($e \geq \log_2 f$) 时为对数级别；当鸡蛋有限（如 $e = 2$ ）时为 $\sqrt{f}$ 级别。

7.3 边界值验证

验证 1： $dp[e][1] = 1$ （任意鸡蛋数，1 层楼）

鸡蛋数	结果
1	1
2	1
5	1
10	1
100	1

验证 2： $dp[1][f] = f$ （1 个鸡蛋，任意楼层数）

楼层数	结果
1	1
5	5
10	10
20	20
50	50

验证 3：单调性

- $dp[e][f]$ 随 f 递增：**PASS**（固定鸡蛋数，楼层数越多需要试验次数越多）
- $dp[e][f]$ 随 e 递减：**PASS**（固定楼层数，鸡蛋越多需要试验次数越少）

7.4 特殊情况：鸡蛋充足

当鸡蛋数足够多时 ($e \geq \log_2(f)$), 问题等价于二分查找:

表 14: 鸡蛋充足时等价于二分查找

鸡蛋数	楼层数	答案	二分查找次数
10	10	4	4
100	100	7	7
1000	1000	10	10
10000	10000	14	14

8 与其他算法的对比

算法对比可视化如图 6 和图 7 所示。

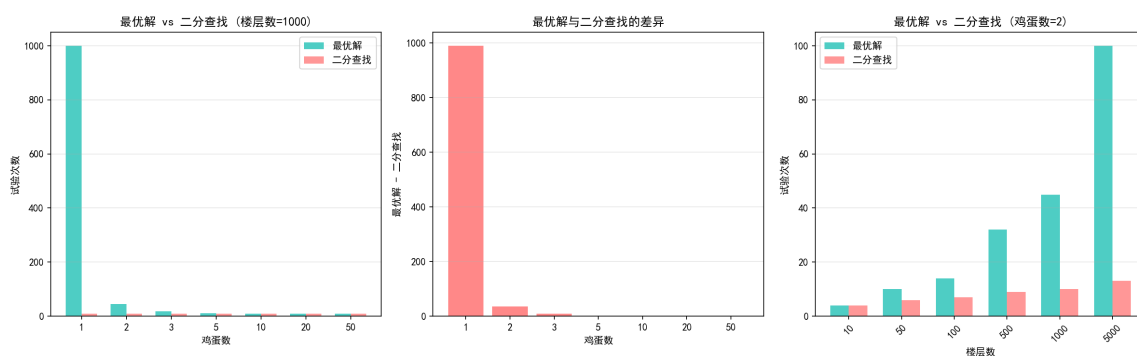


图 6: 最优解 vs 二分查找

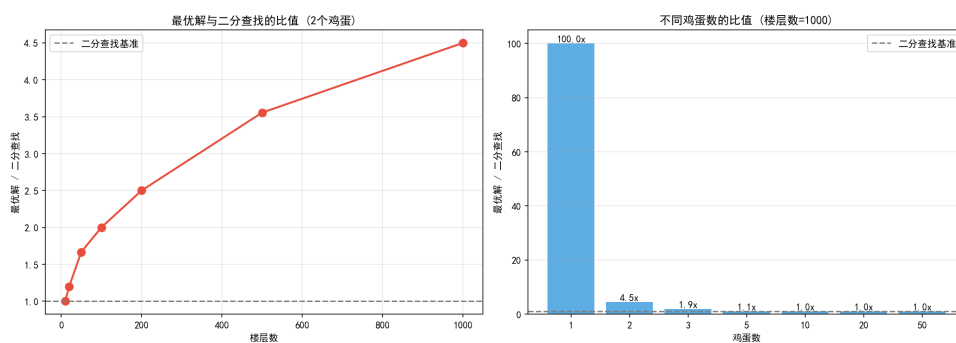


图 7: 贪心策略 vs 最优策略

8.1 朴素二分查找

表 15: 与朴素二分查找对比

鸡蛋	楼层	最优解	二分查找	说明
1	10	10	4	1 个鸡蛋，必须逐层
2	10	4	4	等于二分查找
5	10	4	4	等于二分查找
1	100	100	7	1 个鸡蛋，必须逐层
2	100	14	7	比二分多 7 次
5	100	7	7	等于二分查找
2	1000	45	10	比二分多 35 次
5	1000	11	10	比二分多 1 次

结论：

- 当鸡蛋充足时，最优解等于二分查找
- 当鸡蛋不足时，最优解需要更多次数

8.2 贪心策略 vs 最优策略

贪心策略的局限性：

贪心策略：每次选择”看起来最优”的楼层（如中间楼层）

最优策略：考虑所有可能的选择，取全局最优

示例：2 个鸡蛋，10 层楼

贪心策略（二分查找）：

- 第 1 次：第 5 层
- 如果碎了：需要在 1-4 层逐个试，最多再试 4 次，共 5 次
- 如果没碎：在 6-10 层继续二分
- 最坏情况：5 次

最优策略（交替 DP）：

- 第 1 次：第 4 层（不是第 5 层！）
- 如果碎了：在 1-3 层逐个试，最多再试 3 次，共 4 次
- 如果没碎：在 5-10 层继续

- 最坏情况：4 次

为什么第 4 层比第 5 层更优？

因为要平衡”碎了”和”没碎”两种情况的最坏代价：

- 第 4 层：碎了 → 3 次，没碎 → 需要在 6 层中找，也是 3 次
- 第 5 层：碎了 → 4 次，没碎 → 需要在 5 层中找，也是 4 次
- 但第 4 层的总次数更少！

8.3 鸡蛋数对结果的影响

表 16: 鸡蛋数对结果的影响（楼层数 = 1000）

鸡蛋数	答案	比值	说明
1	1000	-	基准（必须逐层）
2	45	22.22x	大幅提升
3	19	2.37x	继续提升
5	11	1.73x	提升减缓
10	10	1.10x	接近极限
20	10	1.00x	达到极限
50	10	1.00x	不再提升
100	10	1.00x	不再提升

结论：鸡蛋数从 1 增加到 2 时收益最大，之后边际收益递减，直到达到二分查找的下限。

8.4 不同算法的适用场景

表 17: 不同算法的适用场景

场景	最优解	二分查找	推荐算法
鸡蛋充足 ($e \geq \log_2 f$)	7	7	二分查找
鸡蛋很少 ($e = 1$)	100	7	必须逐层尝试
鸡蛋适中 ($e = 2$)	14	7	交替 DP
鸡蛋较多 ($e = 10$)	10	10	交替 DP

9 算法正确性证明

9.1 最优子结构

鸡蛋掉落问题具有最优子结构性质：

- 原问题（ e 个鸡蛋， f 层楼的最少试验次数）可以分解为子问题
- 在第 x 层扔鸡蛋后，无论碎或不碎，都形成一个规模更小的子问题
- 原问题的最优解包含子问题的最优解

9.2 无后效性

当前状态只与鸡蛋数和楼层数有关，与之前的操作历史无关。

9.3 重叠子问题

在递归求解过程中，相同的 (e, f) 状态会被多次计算，因此适合使用动态规划。

10 实验总结

10.1 主要收获

1. 掌握了动态规划的基本思想：通过子问题的最优解构建原问题的最优解
2. 理解了状态定义的重要性：不同的状态定义会导致截然不同的算法效率
3. 学会了算法优化技巧：二分查找优化、状态转换等
4. 掌握了算法复杂度分析方法：理论分析与实际测试相结合

10.2 关键发现

1. 交替 DP 是最优解法：时间复杂度 $O(e \times \text{answer})$ ，空间复杂度 $O(e)$
2. 二分查找优化效果显著：在基础 DP 上可获得约 129 倍加速
3. 算法可处理大规模数据：交替 DP 可在 1 秒内处理 10^{12} 层楼（万亿级）

10.3 算法对比总结

鸡蛋掉落问题 - 算法对比总结

算法	时间复杂度	空间复杂度	适用场景	10蛋1000层耗时
基础DP	$O(e \times f)$	$O(e \times f)$	教学演示	~1000ms
记忆化搜索	$O(e \times f)$	$O(e \times f)$	递归思维	~1000ms
二分优化DP	$O(e \times f \times \log(f))$	$O(e \times f)$	一般场景	~3ms
交替DP	$O(e \times \text{answer})$	$O(e)$	最优解法	~0.007ms
组合数方法	$O(\text{answer} \times \min(e, \text{answer}))$	$O(1)$	数学推导	~0.01ms

图 8: 算法对比总结表

表 18: 算法对比总结

算法	时间复杂度	空间复杂度	适用场景
基础 DP	$O(e \times f^2)$	$O(e \times f)$	教学演示
记忆化搜索	$O(e \times f^2)$	$O(e \times f)$	递归思维
二分优化 DP	$O(e \times f \times \log f)$	$O(e \times f)$	一般场景
交替 DP	$O(e \times \text{answer})$	$O(e)$	最优解法
组合数方法	$O(\text{answer} \times \min(e, \text{answer}))$	$O(1)$	数学推导

10.4 心得体会

鸡蛋掉落问题是一个经典的动态规划问题，通过这个问题的学习，我深刻理解了：

- 1. 动态规划的核心是状态定义和状态转移
- 2. 不同的状态定义可以带来数量级的效率差异
- 3. 算法优化需要深入理解问题的本质特性（如单调性）
- 4. 理论分析和实际测试相结合才能全面评估算法性能

11 附录：完整源代码

详见 `egg_drop.py` 文件。

Listing 1: Main Function List

```
1 # 1. egg_drop_dp()           - Basic DP
2 # 2. egg_drop_memo()         - Memoization Search
3 # 3. egg_drop_dp_optimized() - Binary Search Optimized DP
4 # 4. egg_drop_dp_alt()       - Alternative DP State
5 # 5. egg_drop_math()         - Binomial Coefficient Method
6 # 6. egg_drop_brute_force()  - Brute Force (for verification)
7 # 7. verify_correctness()    - Correctness Verification
8 # 8. performance_test()      - Performance Test
9 # 9. find_max_scale()        - Max Scale Test
10 # 10. space_analysis()       - Space Analysis
11 # 11. complexity_analysis()  - Complexity Analysis
12 # 12. efficiency_improvement_analysis() - Efficiency Analysis
```